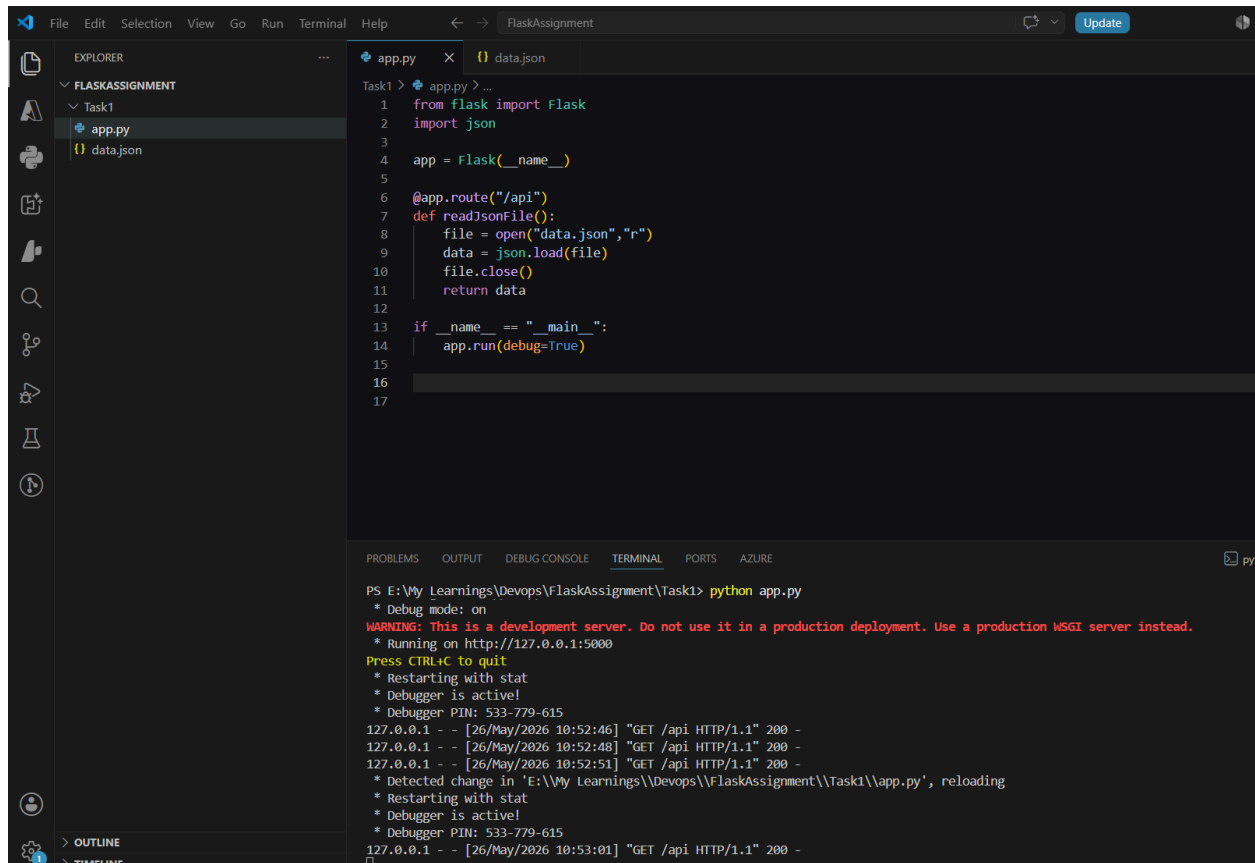


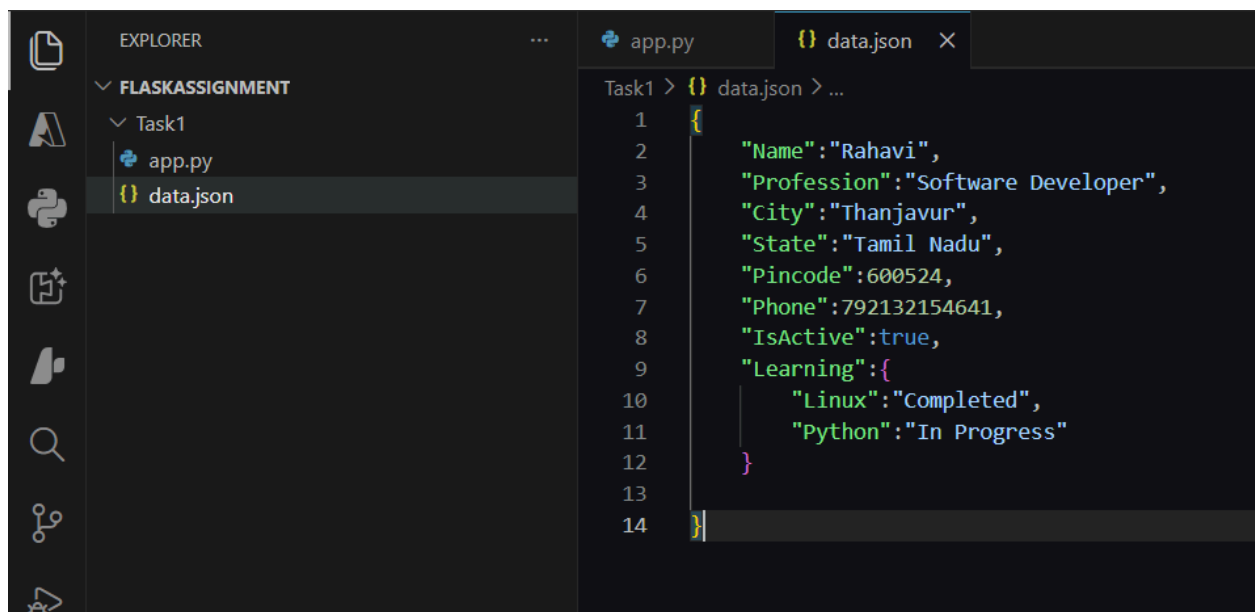
1. Create a Flask application with an `/api` route. When this route is accessed, it should return a JSON list. The data should be stored in a backend file, read from it, and sent as a response.



The screenshot shows the Visual Studio Code editor with a Flask application. The Explorer pane on the left shows a project named 'FLASKASSIGNMENT' with a subdirectory 'Task1' containing 'app.py' and 'data.json'. The main editor displays the code for 'app.py'.

```
1 from flask import Flask
2 import json
3
4 app = Flask(__name__)
5
6 @app.route("/api")
7 def readJsonFile():
8     file = open("data.json", "r")
9     data = json.load(file)
10    file.close()
11    return data
12
13 if __name__ == "__main__":
14     app.run(debug=True)
```

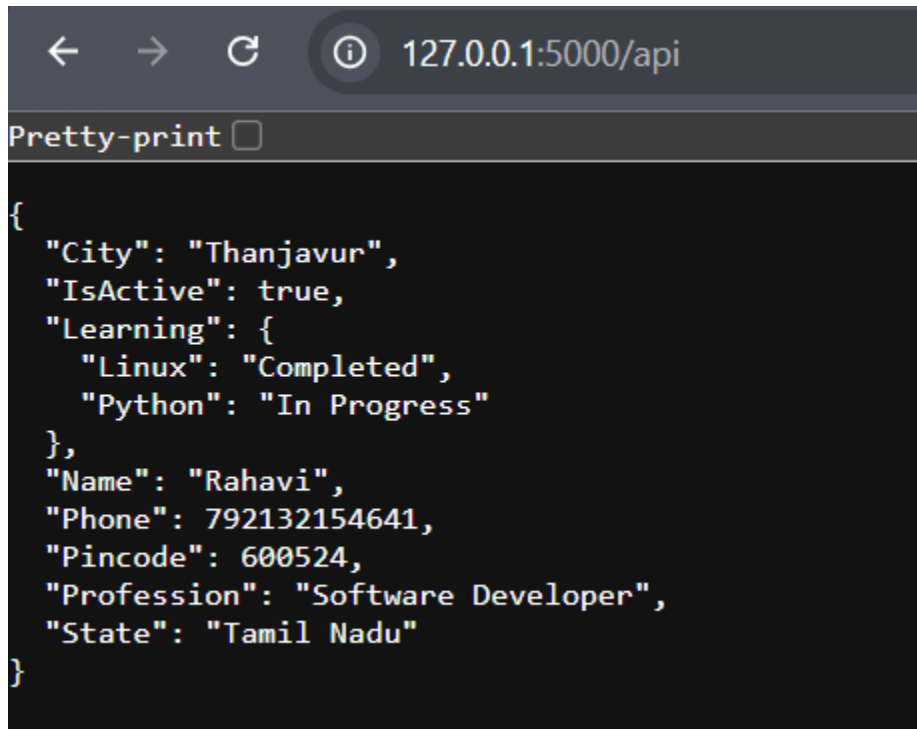
The terminal at the bottom shows the command `python app.py` being executed. The output indicates that the application is running on `http://127.0.0.1:5000` and is in debug mode. It also shows several GET requests to the `/api` endpoint, all returning a 200 status code. A message indicates that a change in 'app.py' was detected and the application is reloading.



The screenshot shows the Visual Studio Code editor with the 'data.json' file open. The Explorer pane on the left shows the same project structure as the previous screenshot. The main editor displays the content of 'data.json'.

```
1 {
2     "Name": "Rahavi",
3     "Profession": "Software Developer",
4     "City": "Thanjavur",
5     "State": "Tamil Nadu",
6     "Pincode": 600524,
7     "Phone": 792132154641,
8     "IsActive": true,
9     "Learning": {
10        "Linux": "Completed",
11        "Python": "In Progress"
12    }
13 }
14 }
```

Output:

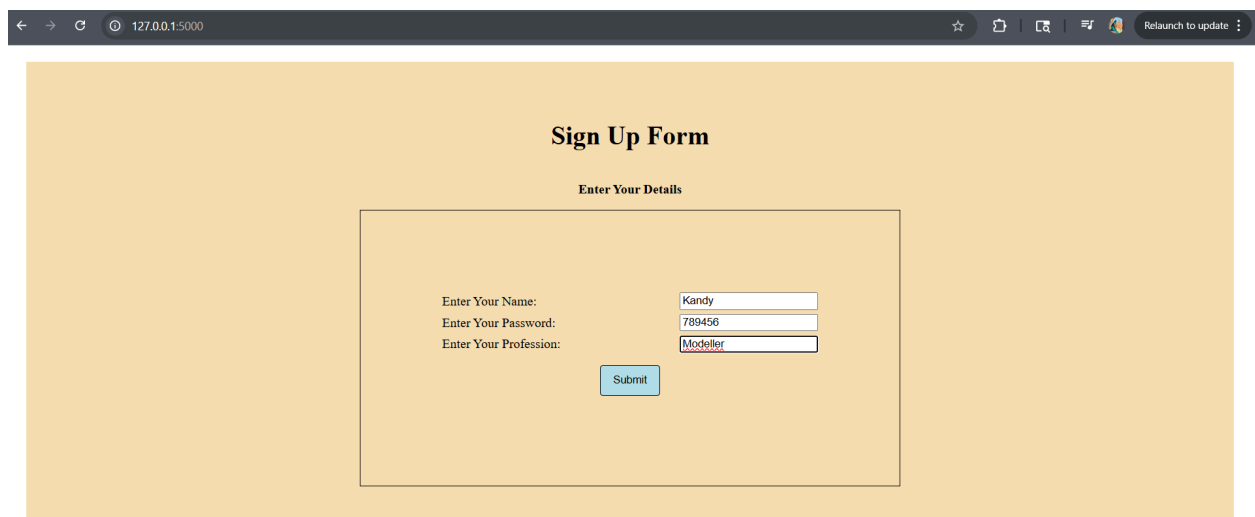


A screenshot of a web browser window. The address bar shows the URL `127.0.0.1:5000/api`. Below the address bar, there is a checkbox labeled "Pretty-print" which is currently unchecked. The main content area of the browser displays a JSON object in a dark-themed code editor. The JSON object represents a user profile with the following fields: "City" (Thanjavur), "IsActive" (true), "Learning" (an object with "Linux" as "Completed" and "Python" as "In Progress"), "Name" (Rahavi), "Phone" (792132154641), "Pincode" (600524), "Profession" (Software Developer), and "State" (Tamil Nadu).

```
{
  "City": "Thanjavur",
  "IsActive": true,
  "Learning": {
    "Linux": "Completed",
    "Python": "In Progress"
  },
  "Name": "Rahavi",
  "Phone": 792132154641,
  "Pincode": 600524,
  "Profession": "Software Developer",
  "State": "Tamil Nadu"
}
```

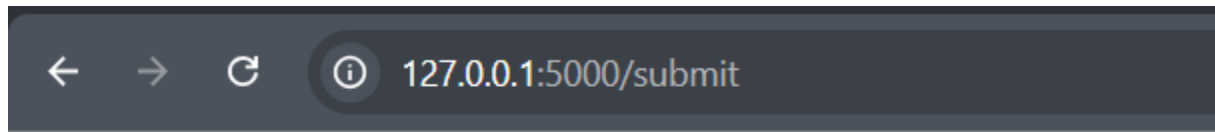
2. Create a form on the frontend that, when submitted, inserts data into MongoDB Atlas. Upon successful submission, the user should be redirected to another page displaying the message **"Data submitted successfully"**. If there's an error during submission, display the error on the same page without redirection.

Form:

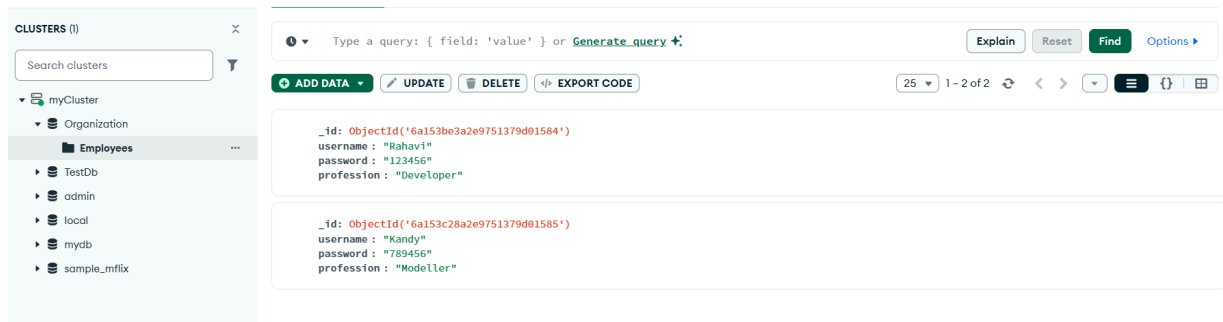


A screenshot of a web browser window showing a "Sign Up Form". The browser's address bar displays `127.0.0.1:5000`. The form is centered on a light orange background. It has a title "Sign Up Form" and a subtitle "Enter Your Details". The form contains three input fields: "Enter Your Name:" with the value "Kandy", "Enter Your Password:" with the value "789456", and "Enter Your Profession:" with the value "Modeller". Below these fields is a blue "Submit" button.

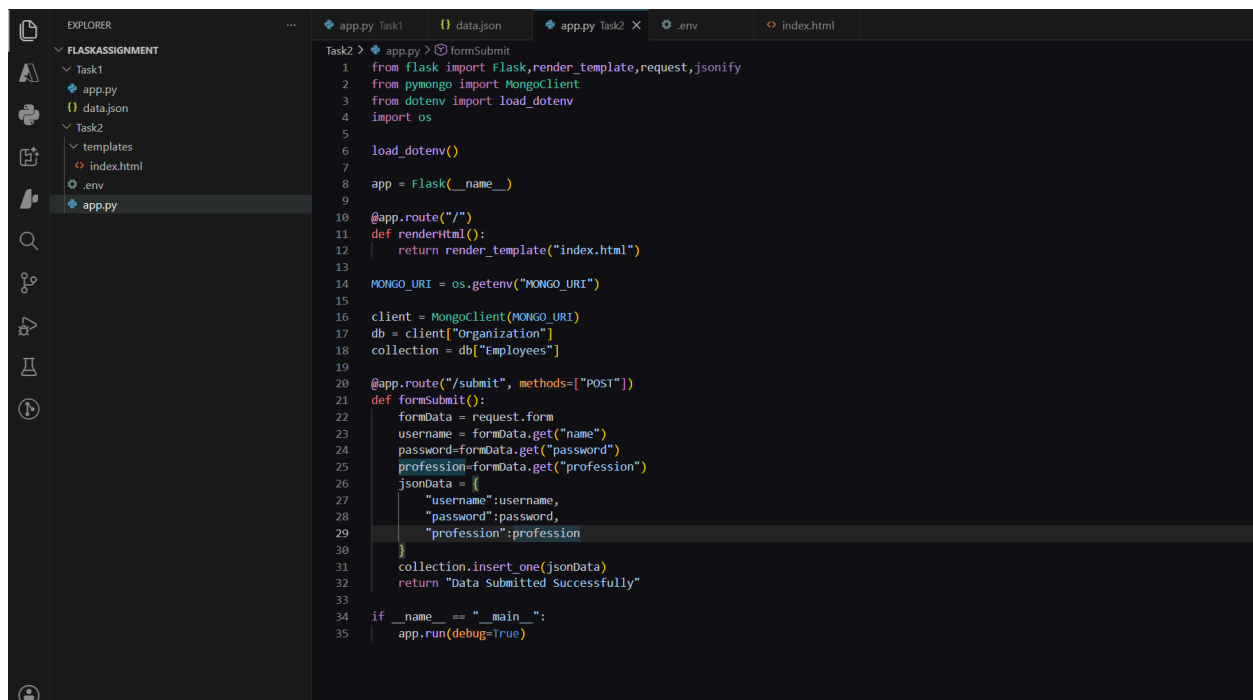
Browser Output:



Mongo DB Output:



Flask Code:



Github Repo Link:

<https://github.com/RahaviS/tutedudeAssignment/tree/main/FlaskAssignment>

Submission Guidelines -: Attach Screenshots or command along with explanation and submit in doc(google doc or microsoft doc) format also attach github repo link